Algoritmi paraleli și distribuți

Masurarea performanțelor sistemelor paralele

+ Parallel STL

Metrici pentru sistemele și aplicațiile paralele

- Timp de execuție (en., *Execution Time*)
- Accelerare (en., *Speedup*)
- Eficiență: se referă la complexitatea algoritmului
- Performanță: se referă la implementare: limbaj, siste, structuri de date, etc.
- Cantitatea de date procesată (en., *System Throughput*)
- Costul
- Granularitatea
- Scalabilitatea
- Gradul de concurență (i.e, paralelizare, utilizare)

Evaluarea programelor secvențiale și paralele

- ***Algoritmii secvențiali*** sunt evaluați din punct de vedere al:
  - Eficienței algoritmice: aceasta este o măsură teoretică
  - Performanței: aceasta se referă la timpul de rulare si depinde de implementare
  - ***Obs***: eficiența nu depinde de un anumit sistem
- ***Timpul de rulare a unui program paralel*** depinde și de alți factori: numărul de procesoare, modul de comunicare din sistemul pe care se rulează.
  - ***Obs***: Algoritmii paraleli trebuie anaizați în contextul platformei (HW, SO, CPU (număr de procesoare), GPU) pe care rulează.
  - Un sistem paralel este combinația dintre algoritmul paralel, implementarea acestuia și platforma pe care se execută aplicația.

Despre performanță

- **Timpul ” Wall clock”** (i.e., timpul real, timpul total necesar) este timpul măsurat de la începutul rularii pe primul procesor până când se termină rularea pe ultimul procesor.
  - Notație: **Ts** este timpul rulării secvențiale; **Tp** este timpul necesar rulării paralele
- **Accelerarea** (en., speedup): Cu cât este mai rapidă varianta paralelă?
  - Care este versiunea de bază (en., baseline/reference) cu care comparăm varianta paralelă? Este cea mai bună/rapidă? Este cea medie?
  - Putem folosi o variantă suboptimală secvențială?
- **Granularitatea**
  - Mică = număr mare de task-uri simple
  - Mare = număr mic de task-uri complexe

Despre performanță (cont.)

- **Gradul de concurență** este numărul maxim de task-uri care pot fi executate simultan.
- **Gradul mediu de concurență** este numărul mediu de task-uri care pot fi executate în paralel pe întreaga rulare a programului.
    - Gradul de concurență crește pe măsură ce descompunerea este mai fină.
- **Calea critică** este costul cel mai mare pe care îl poate atinge un drum de la *start* la *finish* în graful de dependințe al task-urilor.
  - Costul căii este suma ponderilor nodurilor.
- **Graful de interacțiune al task-urilor**
  - Un nod corespunde unui task, iar o muchie conectează două noduri daca task-urile respective cumunică/interacționază unul cu altul.

Costuri suplimentare (en., *overheads*)

- Dacă avem la dispoziție două procesoare trebuie ca varianta paralelă să ruleze de două ori mai repede?
- NU! Deoarece apar costuri suplimentare:
  - Calcule inutile
  - Așteptare/pauză (en., *idle*)
  - Timp necesar comunicației
  - Diverse dispute (ex., *data race*)

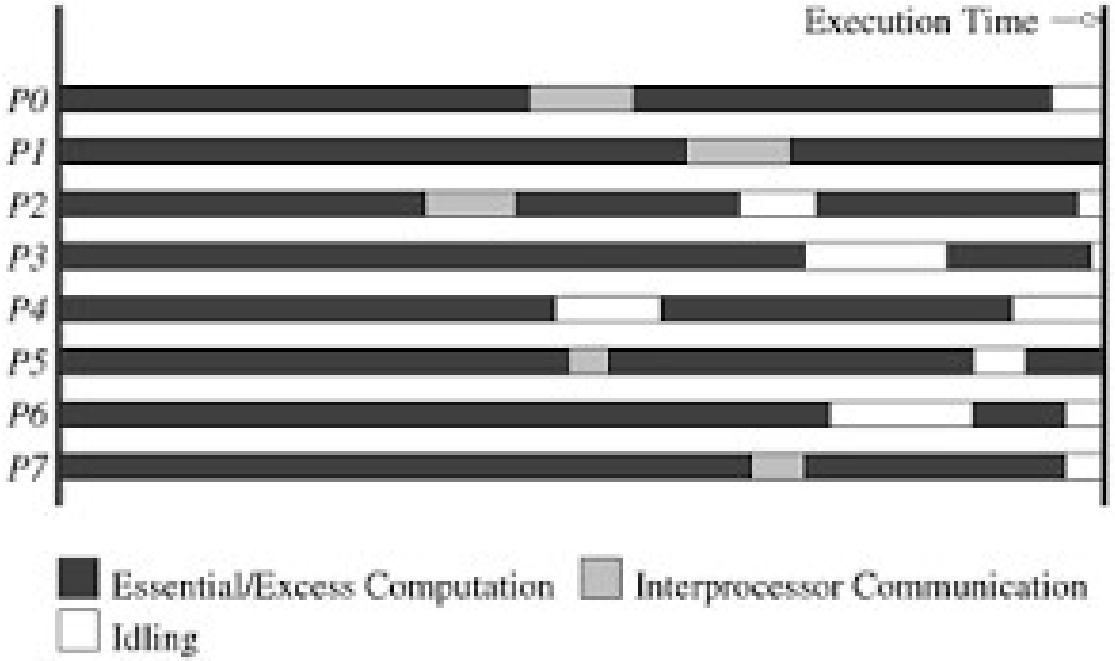
... care conduc la degradarea performanței.

Timpul de execuție și costurile suplimentare

- **Timpul de execuție/răspuns** măsoară intervalul de timp scurs între trimiterea unei cereri până când este primit raspunsul.
- **To** = costul suplimentar (en., *overhead*) pentru varianta paralelă

To = p*Tp – Ts, unde Tp si Ts sunt timpii pentru execuția paralelă și secvențială.

Execuția unui program paralel pe 8 procesoare. Observați stările unui procesor: *rulare*, *comunicare*, *așteptare*.



Minimizarea costurilor suplimentare

- Sursele tradiționale ale costurilor suplimentare sunt:
  - Apelurile de funcții: necesită lucrul cu stiva, apar ramuri ale programului.
  - Recursivitate
- Pentru soluții algoritmice diferite ale aceleiași probleme putem avea costuri suplimentare de paralelizare diferite
- Costurile suplimentare necesare implementării unei anumite soluții algoritmice pot decide dacă are rost sau nu să paralelizăm algoritmul/funcția respectivă.

Surse ale costurilor suplimentare

- **Comunicarea între procese** apare atunci când este necesară transmiterea de date/rezultate de la un procesor la altul.
- **Așteptarea/pauza** (en., *idle*) poate apărea la un proces din cauza lipsei balansării încarcării (en., *load balancing*) sau a sincronizărilor necesare.
- **Calculele în plus** (en., *excess computation*) sunt acele calcule care nu apar în varianta secvențială.
  - Acestea sunt multe atunci când algoritmul este dificil de paralelizat sau când anumite calcule sunt efectuate pe mai multe procesoare pentru a diminua comunicarea.

Accelerarea (en., *speedup*)

- **Acelerarea** este cea mai utilizată măsură a performanței versiunii paralele a unui algoritm secvențial.
- **Ts** este cel mai bun timp secvențial.
- **Tp** este timpul necesar rulării versiunii paralele pe **p** procesoare.

Accelerarea = Ts / Tp

- Accelerarea liniară apare când Tp = Ts / p, ceea ce reprezintă cazul ideal.
- Cum se determină Ts?
  - Pe un procesor dintr-un sistem paralel?
  - Folosind cel mai bun sistem secvențial disponibil?
  - Rulând un algoritm paralel pe un singur procesor?

Accelerarea: altă definiție/abordare

- **Accelerarea** este timpul necesar unui algoritm paralel să ruleze pe un procesor împărțit la timpul necesar algoritmului paralel sa ruleze pe N procesoare.
  - Această abordare este înșelătoare deoarece multi algoritmi paraleli folosesc operații adiționale (ex., comunicarea) pentru a obține paralelismul.
  - Aceasta abordare mărește pe Ts, și astfel exagerează accelerarea determinată.
- ***Exemplu***: Bubble sort

Ts = 150 s

Tp = 30 s ;// pentru o variantă paralelă eficientă

Accelerarea = 150 / 30 = 5. Este corect?

Dacă un quicksort serial durează numai 20 s?

Atunci accelerarea = 20/30 = 0.67 ☹ Aceasta este o evaluare mai realistă!

Limitele accelerării

- Accelerarea poate fi 0, adică programul paralel nu se termină niciodată.
- Accelerarea este limitată de ***p***, numărul de procesoare.
- O accelerare mai mare decât *p* este posibilă numai dacă fiecare element de procesare (ex., procesor) petrece mai puțin de *Ts/p* timp pentru rezolvarea problemei.
  - În această situație, un singur procesor ar putea fi folosit pentru a obține un program secvențial mai rapid, ceea ce contrazice presupunerea că avem la dispoziție un program secvențial optim ca referință (en., *baseline*).

Factori care limitează accelerarea

- Costurile suplimentare de calcul (software)
  - Chiar dacă avem la dispoziție un algoritm paralel corect și complet echivalent, costurile suplimentare apar de obicei din cauza implementării concurente.
- Echilibrarea proastă a încărcării (en., *load balancing*)
  - Accelerarea este de obiei limitată de viteza celui mai încet nod. Astfel, un aspect esențial se referă la asigurarea încărcării egale a nodurilor (ex., procesoare) cu același efort de calcul.
- Costurile suplimentare datorate comunicării
  - Presupunând ca procesarea și comunicarea nu pot fi suprapuse (i.e., procesăm apoi transmitem rezultatul) putem spune că timpul necesar comunicării de date între procesoare degradează în mod direct accelerarea.

Factori care limitează accelerarea (cont.)

- Cod care este nativ serial
  - Dacă dorim să paralelizăm o aplicație, dar asta nu înseamnă că trebui ca tot codul să aibă proprietatea că poate fi paralelizat.
- Algoritmi care nu sunt optimi
  - Optimalitatea algoritmului proiectat crează premisele unei implementări performante.
- Costurile suplimentare datorate sincronizărilor
  - Nu putem paraleliza un algoritm fără să fie nevoie de sincronizare.

Accelerarea liniară

- Accelerarea liniară reprezintă dezideratul paralelizării. Astfel, folosind **N** procesoare, o accelerare de **N** este considerată excelentă.
- În practică, accelerarea este mai mică decât N.
- Pentru aplicațiile care scalează bine accelerarea ar trebui să crească proporțional cu numărul de procesoare.
  - Când spunem ca o aplicație scalează?
- Accelerarea supra-liniară apare atunci când:
  - Sunt folosite valori injuste pentru Ts
  - Apar diferențe în ceea ce privește natura HW folosit.

Accelerarea supra-liniară bazată pe resurse

- Dacă avem la dispoziție lățime de bandă mai mare pentru accesul la cache atunci putem obține accelerare supra-liniară.
- ***Exemplu***:
  - Un procesor cu 64 KB memorie cache face ca în 80% din situații datele necesare să se afle în cache.
  - Dacă sunt folosite două procesoare, atunci în 90% din situații datele necesare se află în cache.
  - Din cele 10% rămase, 8% se gasesc în memoria locală iar 2% sunt la distanță.
  - Dacă timpii de acces la cache, memoria locală si la distanță sunt de 2ns, 100ns și 400ns, atunci accelerarea va fi de 2.43!

Eficiența

- Accelerarea nu măsoară/determină/stabilește cât de eficient sunt utilizate procesoarele.
  - Merită sa folosim 100 de procesoare pentru o accelerare de 2?
- Eficiența este definită ca raportul dintre accelerare și numărul de procesoare necesar pentru a obține acea accelerare.

$$e = \text{accelerarea} / \text{numarul de procesoare} = s / p$$

- Eficiența este maxim 1 și reprezintă fracțiunea de timp în care un procesor este folosit la maxim.
  - Ideal, daca s = p atinci e =1.

Redundanța

- Redundanța HW: mai multe procesoare sunt folosite pentru o singură aplicație, dintre care cel puțin unul nu rulează nimic (i.e., *standby*)
  - Crește costurile dar, de multe ori, este o soluție practică pentru evitarea întreruperii rulării aplicației
  - Sistemele dedicate au căi de comunicație redundante. Astfel, chiar dacă o cale de comunicație (i.e., magistrală) este distrusă fizic totuși aplicația va continua să ruleze. În această situație ne îndreptăm către un sistem distribuit!
- Redundanța poate fi implementată la nivele diferite
  - Servere individuale pot fi replicate
  - HW-ul redundant poate fi folosit pentru activități cu prioritate mai scăzută atunci când nu sunt detectate defecte (en., *failures*)
- Redundanța SW: proiectarea sw trebuie realizată astfel încât starea aplicației să poată fi recuperată atunci când sunt detectate defectări (en., *faults*)

Granularitatea paralelismului

- Este determinată de dimensiunea medie a componentelor secvențiale care compun calculul paralel
- **Paralelismul independent** este atunci când procesele sunt independente și nu este nevoie de sincronizare.
- **Paralelismul este**
  - **Slab granularizat** (en., *coarse-grained*) atunci când procesele sunt relativ independente iar sincronizările sunt ocazionale.
  - **Mediu granularizat** atunci când firele de execuție se sincronizează frecvent.
  - **Fin granularizat** (en., *fine-grained*) atunci când este nevoie de sincronizare la fiecare câteva instrucțiuni.

Granularitatea: efecte asupra performanței

- De obicei, folosirea unui număr mai mic de procesoare crește performanța.
  - Diminuarea scalării (en., *scaling down*) unui sistem paralel apare atunci când sunt folosite mai puține procesoare decât maximul posibil.
- Atunci când numărul de procesoare scade cu n/p, timpul procesor crește la fiecare cu n/p.
- Costul de comunicare nu ar trebui să crească cu această cantitate.
- Aceasta este o posibilă îmbunătățire oferită de administrarea granularității.

Granularitatea: efecte asupra performanței

- Folosirea unui număr foarte mare de procesoare/fire (granularitate fină) poate conduce la diminuarea masivă a paralelismului.
  - Ex. Înmulțirea unei matrici dense cu un vector nu poate crea mai mult de n^2 task-uri.
- Task-urile concurente trebuie uneori să schimbe date cu alte task-uri. Această situație conduce la costuri suplimentare de comunicație.

Concluzie:

- Compromisul dintre granularitatea descompunerii și costurile suplimentare asociate determină limitele de performanță ale versiunii paralele.

Scalarea programelor paralele

- Eficiența unui program paralel este

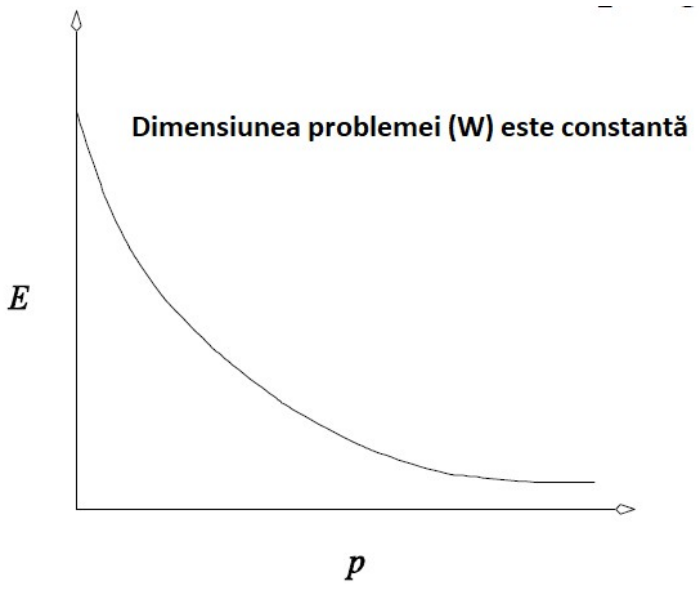
  $E = S/p = Ts / (p*Tp)$ sau $E = 1/(1 + Tp/Ts)$
- Costurile suplimentare (en., *overheads*) totale **To** sunt o funcție crescătoare dependentă de **p**
- Pentru o anumită dimensiune a problemei (**Ts** este constant), pe masură ce creștem numărul de procesoare va crește și **To**.
- Astfel, eficiența generală a programului scade!
- **Soluție**: Trebuie determinată granularitatea optimă pentru problema abordata. Aceasta se determină experimental în funcție de platforma pe care rulează aplicația.

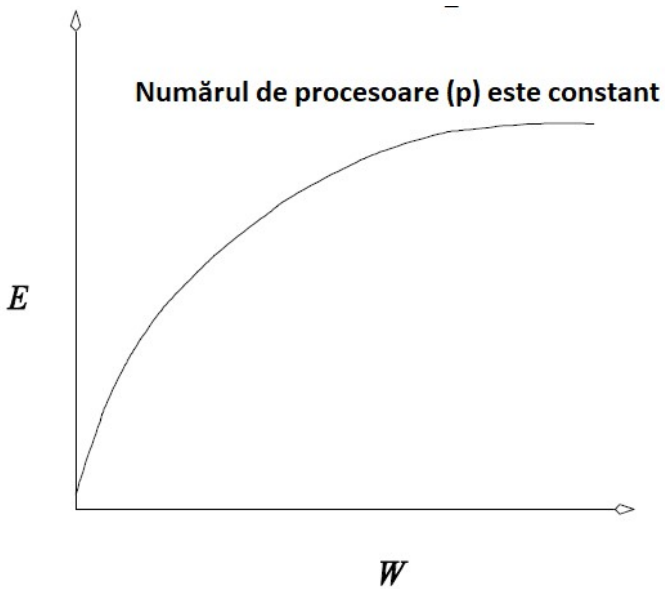
Scalarea programelor paralele (cont.)

- Asadar, costurile suplimentare totale **To** depind atât de dimensiunea problemei **Ts** cât și de numărul de procesoare **p**.
- În multe cazuri, **To** crește subliniar în raport cu **Ts**.
  - În aceste cazuri, eficiența crește dacă dimensiunea problemei este mărită dar păstrăm numărul de procesoare constant.
- Sistemele care au aceste proprietăți sunt denumite sisteme paralele **scalabile**.

Eficiența (E) în raport cu numarul de procesoare (p) și dimensiunea problemei (W)

- Cresterea numărului de procesoare descrește eficiența pentru o anumită dimensiune a problemei
- Cresterea dimensiunii problemei poate crește eficiența pentru un anunmit număr de procesoare.





Determinarea scalabilității

- Dimensiunea problemei W este dată de numărul de operații care sunt efectuate de cel mai bun algoritm secvențial care rezolvă problema.
- Gradul/raportul cu care poate fi crescută dimensiunea problemei în raport cu numărul de procesoare pentru a păstra eficiența constantă determină scalabilitatea sistemului.
- Cu cât raportul este mai mic cu atat mai bine.

Variația gradului de concurență/paralelism

- Este dată de numărul de operații care pot fi executate simultan/paralel.
- Pentru paralelismul de tip pipeline (unde datele au forma unui vector) gradul este același cu numărul de procesoare.
- Gradul de concurență poate fi constant în timpul rulării algoritmului, dar de cele mai multe ori variază.
- Metoda cea mai intuitivă pentru reprezentarea prelucrarilor paralele este cu ajutorul grafurilor computationale (en., *computation graphs*) care utilizează grafuri directionate aciclice în care sunt definite nodurile sursă și destinație.

Alte metrici ale scalabilității

- Sunt impuse de nevoile specifice ale aplicațiilor
- Pentru aplicațiile în timp-real obiectivul este ca scalarea să permită îndeplinirea task-ului într-o anumită limită de timp.
- Sistemele care au constângeri în ceea ce privește memoria, metricile iau în modul în care evoluează necesitatea de memorie în raport cu dimensiunea problemei sau numarul de procesoare folosit.

Accelerarea scalată (en., *scaled speedup*)

- Este acea accelerare care se obține atunci când dimensiunea problemei este crescută liniar în raport cu numărul de procesoare.
- Dacă accelerarea scalată este liniară atunci considerăm că sistemul este scalabil.
- Dacă memoria necesară crește liniar în raport cu **p**, atunci accelerarea scalată crește dimensiunea problemei până la umplerea memoriei.
- Alternativ, dimensiunea problemei se poate crește până când timpul de execuție atinge un maxim permis.

Legea lui Amdahl

- Este folosită pentru a detrmina limita superioară a accelerării.
- Accelerarea unui algoritm paralel este limitată de procentul de operații care trebuie efectuate secvențial.
- Legea lui Amdahl presupune că datele de intrare au dimensiune fixă/constantă și același de operații seriale sunt necesare.
  - Dacă timpul necesar calculelor seriale este o fracție **σ** din timpul total (0< **σ** <= 1) atunci timpul de calcul paralel este (1- **σ**)
- Presupunând că accelerarea este liniară
  - Tserial = σ * T1
  - Tparalel = (1- σ) * T1 / N;// N este numărul de procesoare
  - Accelerarea = 1 / (σ + (1- σ) / N)

Legea lui Amdahl

- Implicații
  - Cantitatea mare de program secvențial limitează performanța obținută prin adăugarea de procesoare
  - Cresterea numarului de procesoare nu se reflecta in caștiguri de performanță
  - Partea secvențială trebuie redusă pentru accelerare maximă

Consecințe ale legii lui Amdahl

- Presupunem că $\sigma = 0.1$ (10% este cod secvential)
  - Cu 2 procesoare: $S(2) = 1/((1-0,9) + 0,9/2) = 1,82$
  - Cu 10 procesoare: $S(2) = 1/((1-0,9) + 0,9/10) = 5,26$
  - Cu un număr infinit de procesoare: $S(\infty) = 1/(1-0,9) = 10$

- Concluzie:
  - De ce să ne complicăm inutil cu procesarea paralelă? ... Putem să așteptăm un procesor mai rapid :)

Condecințe ale legii lui Amdahl

- Pentru a obține o accelerare liniară cu 100 de procesoare (i.e, accelerarea să fie 100) trebuie ca **nici un calcul** sa nu trebuiască să fie facut secvențial.
- Pentru a avea o accelerare de 99 cu 100 de procesoare trebuie ca maxim 0.01% din programul inițial trebuie să se evecute secvențial.
- Care este accelerarea pe care o putem obține cu 100 de procesoare dacă 30% din programul original ramane secvențial?

$$\text{Accelerarea} = 1/(0.7 + 0.7/100) = 1.4$$

- **Concluzie**: Partea secvenială trebuie restructurată pentru a permite paralelizare eficientă.

Limitări ale legii lui Amdahl

- Pentru a evita limitările legii lui Amdahl trebuie să paralelizăm algoritmii care au parte de prelucrare secvențială mică.
- Legea lui Amdahl nu ia în caclul costurile suplimentare (en., *overheads*) necesare pentru comunicare, sincronizare, administrarea firelor de execuție.
- Legea lui Amdahl nu ia în caclul dimensiunea problemei.
  - De obicei, pe măsură ce numărul de procesoare crește este probabil ca și dimensiunea datelor procesate să crească.

Legea lui Gustafson (accelerarea scalată)

- Dacă o aplicație paralelă care folosește 32 de procesoare poate prelucra date de intrare de 32 de ori mai mari decât datele de intare inițiale, crește și timpul de execuție al parții secvențiale din algoritmul paralel?
  - NU, nu crește cu același raport ca și creșterea dimensiunii datelor de intare.
  - Exeprimentele arată ca timpul de execuție secvențial va rămâne aproape constant.
- **Accelerarea <= p + (1-p)*s**, unde *p* este numărul de procesoare iar *s* este procentul de execuție secvențială dintr-o aplicație paralelă pentru un anumit input.
- Deoarece procentul de execuție secvențială în cadrul aplicației paralele este cunoscut, formula poate fi folosită cu succes pentru determinarea accelerării scalate în raport cu varianta secvențială pentru aceeași dimensiune a problemei.

Rezultate comparative

- Daca 1% din timpul de execuție pe 32 de procesoare este alocat execuției seriale, accelerarea pentru aceleași date de intrare rulate pe un singur procesor cu un singur fir de execuție este:

$$\text{Accelerarea} <= 32 + (1-32)*0.01 = 32 - 0.31 = 31.69$$

- În situația în care procentul de execuție secvențială este de 1% atunci Legea lui Amhdal spune că:

$$\text{Accelerarea} <= 1/(0.01 + (0.99/32)) = 24.43$$

  - Acesta este un calcul nerealist deoarece procentul de timp secvențial este relativ în ceea ce privește execuția pe cele 32 de procesoare

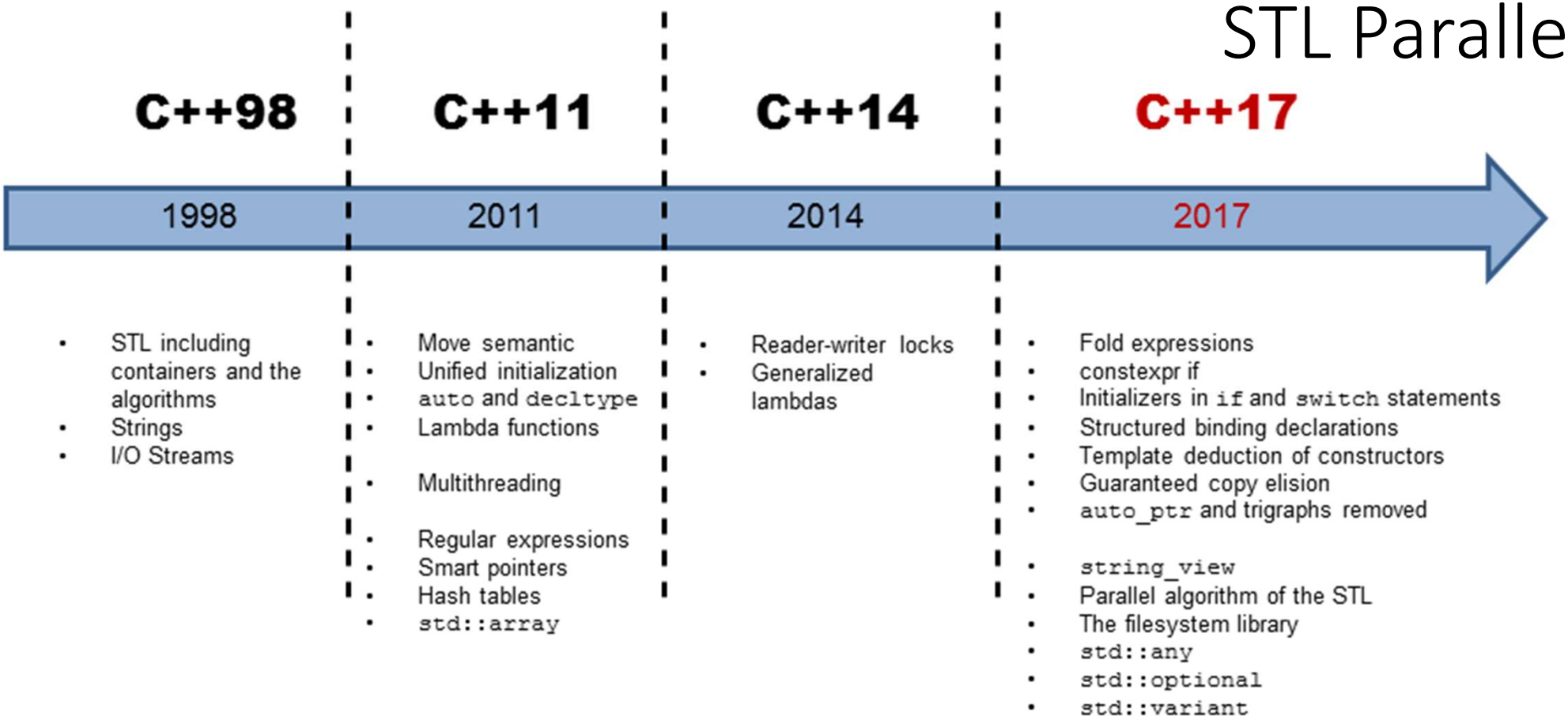
Performanța referitoare la memorie

- **Capacitatea**: De câti bytes este nevoie?
- **Lațimea de bandă**: Câți bytes se pot transfera pe secundă?
- **Latența**: De cât timp este nevoie pentru a prelucra/aduce (en., *fetch*) un cuvânt?
- **Întârzierile de routare**: Apar atunci când datele trebuie aduse din zone diferite de memorie.
- Eventualele dispute se rezolvă prin blocarea memoriei.

STL Parallel

- Biblioteca STL are peste 100 de algorimi pentru căutare, numărare, etc.
- C++ 17 a adăugat suport pentru 69 de algoritmi paraleli în biblioteca standard
  - MSVC a adaugat suport pentru unii algoritmi
- API-ul descris în standard pentru algoritmii paraleli nu precizează exact cum este realizată paralelizarea.
- Algoritmii paraleli se bazează în intregime pe implementarea din bibliotecă și nu necesită nici un ajutor special de la compilator.

STL Parallel



https://www.modernescpp.com/index.php/c-17-new-algorithm-of-the-standard-template-library

STL Parallel: Pași de urmat

1. Găsiți un algoritm pe care doriți sa-l paralelizați în aplicația secvențială. De exemplu, algorimii care necesită mai mult de O(n) timp (ex., sort) sunt candidati foarte buni. De asemenea, trebuie ca aceștia să fie printre cei mai mari consumatori de timp din aplicație.
2. Verificați dacă ceea ce doriți să paralelizați este sigur d.p.d.v. al consistenței. Paralelizarea trebuie să nu permită comportamente nedefinite (en., *undefined behaviour*) sau să conducă la nedeterminism.
3. Alegeți o politică de execuție paralelă.

STL Parallel: Pași de urmat (cont.)

4. #include <execution> pentru a face politicile de execuție paralelă disponibile.

5. Adăugați una dintre politicile de execuție paralelă ca prim parametru al apelului la algoritmului paralel.

6. Rulați/experimentați și realizați un benckmark ar rezultatului pentru a vă asigura că versiuena paraleă este mai rapidă.

Obs:

- Paraleizarea NU este întotdeauna mai rapidă! Mai ales pentru iteratorii care nu au acces aleator (en., *non-random-access iterators*), atunci când dimensiunea datelor de intare este mică, sau atunci când paralelismul determină blocarea pentru accesul la alte resurse (ex., un disk).

STL Parallel

- Algoritmii paraleli depind de diponibilitatea paralelismului HW
- Nu trebuie foarte multe procesoare pentru a demonstra accelerarea.
- Mulți algoritmi sunt de tip *divide et impera* care nu vor scala perfect cu numărul de fire, dar se vor executa mai repede.
- Algoritmii sunt paralelizabili în siguranță dacă funcțiile de acces ale elementelor (ex., operații cu iteratori, predicate, sau alt cod necesar) respectă regulile de acces la date (i.e, *data race*), adică putem avea oricâte citiri ale aceleiași variabile dar nu putem avea decât o scriere.

STL Parallel: politici de execuție

- Standardul include următoarele politici:

Politica secvențială: std::execution::seq

Politica paralelă (dar nu vectorizată): std::execution::par

Politica paralelă și vectorizată: std::execution::par_unseq

Politica vectorizată (dar nu paralelă) [din C++20]: std::execution::unseq

În plus față de cerințele necesare policii paralele, politica paralelă și vectorială necesită ca funcțiile de acces la elemente să accepte progresul/rularea aplicației cu constrângeri mai slabe (ex., nu necesită și nu folosesc lacăte (en., *locks*) pentru acces la memorie). Mai mult decât atât, trebuie ca CPU și sistemul de operare să suporte instrucțiuni SIMD.

Efortul depus pentru paralelizare este minim = adăugarea unui parametru!

STL Parallel: politici de execuție (cont.)

- Biblioteca poate ignora complet politica de execuție specificată
  - Adică poate recurge la execuția serială
- Politica de execuție specificată reprezintă o indicație (en., *hint*) pentru bibliotecă referitor la nivelul maxim de paralelixare/vectorizare acceptat.
- **Limitări**: Biblioteca realizează paralelizarea algoritmului, NU concurența!
  - Concurența este o problemă de proiectare! VS. Paralelismul adresează eficiența la rulare.
  - Rob Pike, Concurrency Is Not Parallelism, https://www.youtube.com/watch?v=oV9rvDllKEg

STL Parallel: politici de execuție (cont.)

- **Limitări (cont.)**:
  - Nu este permisă/posibilă proiectarea concurentă globală a aplicației.
  - Sunt permise doar optimizări locale prin paralelizarea unor apeluri.
  - ***Concluzie gresită***: Folosim STL paralel și nu mai trebuie să acordăm atenție problemelor cauzate de concurență deoarece biblioteca s-a ocupat de acest lucru!
  - Scopul bibliotecii NU este să rezolve probolemele de concurență, ci să rezolve probleme locale de eficiență.

STL Parallel

- Paralelizarea are și costuri suplimentare (en., *overhead*) și va conduce la ineficiență (i.e., timp mai mare de execuție decât varianta secvențială) pentru valori suficient de mici ale lui N (i.e., numarul de numere pe care le sortăm).
- Atunci când se folosește o politică de execuție în paralel este responsabilitatea programatorului să evite situațiile de *data race* sau de blocare (en., *deadlocks*).

STL Parallel

- În timpul execuției unui algortm paralel care foloseste oricare dintre politicile de paralelizare, dacă apelarea unei funcții de acces la un element se finalizează printr-o excepție netratată atunci std::terminate este apelat automat.
- Implementările client pot defini alte politici de execuție care să trateze excepțiile în mod proprie/particular.
- Aruncarea (și tratarea excepțiilor) este recomandată!

STL Parallel: Exemplu

- Filtrarea elementelor dintr-un container
- Se dă un container A cu elemente. Scrieți un program care copiază elementele care corespund unui predicat în containerul B, și returnați containerul B.
- Funcția ***Filter*** are urmatoarea semnătură:

```
auto Filter(const Container& cont, UnaryPredicate p) {}
```

STL Parallel: Exemplu

```
//raw loops
template <typename T, typename Pred>
auto FilterRaw(const std::vector<T>& vec, Pred p) {
    std::vector<T> out;
    for (auto&& elem : vec)
        if (p(elem))
            out.push_back(elem);
    return out;
}
```

STL Parallel: Exemplu

```
//variantă paralelă
std::mutex mut;
std::for_each(std::execution::par, begin(vec), end(vec),
    [&out, &mut, p](auto&& elem) {
      if (p(elem))
      {
        std::unique_lock lock(mut);
        out.push_back(elem);
      }
    });
```

https://www.cppstories.com/2021/filter-cpp-containers/

Pentru alte 10 implementări diferite!

Referințe

- https://www.bfilipek.com/2017/08/cpp17-details-parallel.html
- https://www.bfilipek.com/2018/11/parallel-alg-perf.html
- Bryce Adelstein Lelbach, C++ Parallel Algorithms and Beyond, CppCon 2016, 2016, https://www.youtube.com/watch?v=Vck6kzWjY88
- Billy O'Neal, Using C++17 Parallel Algorithms for Better Performance, Microsoft C++ Team Blog, 2018, https://devblogs.microsoft.com/cppblog/using-c17-parallel-algorithms-for-better-performance/
- Lucian Radu Teodorescu, A Case Against Blind Use of C++ Parallel Algorithms, https://accu.org/journals/overload/29/161/overload161.pdf